Taller — Pilas, colas y listas enlazadas

SD1004 · Estructura de Datos — Institución Universitaria Pascual Bravo
Docente: Juan Duque · Semestre 2026-II
Fecha límite de entrega: domingo 11 de octubre de 2026

ESTUDIANTE: SEBASTIAN BEDOYA BEDOYA

Propósito

Representar y resolver operaciones básicas con pilas, colas, listas enlazadas simples, circulares y dobles. En cada ejercicio explica cómo cambia la estructura y justifica por qué esa estructura es adecuada para el caso.

Instrucciones generales

- Responde en este documento o en un archivo aparte de Word, Markdown o PDF.
- No necesitas programar en un lenguaje específico. Usa pseudocódigo, diagramas y explicaciones con tus propias palabras.
- En los diagramas, identifica el inicio y el final de cada estructura, los enlaces entre nodos y, cuando aplique, el tope, el frente y el final.
- Justifica tus respuestas. Una respuesta final sin mostrar el razonamiento no obtiene el puntaje completo.
- Si dibujas a mano, toma una foto legible e inclúyela en la entrega.
- Fecha y plataforma de entrega: domingo 11 de octubre de 2026, por el medio indicado en el aula del curso.

Parte 1 — Reconocer la estructura adecuada

Para cada situación, selecciona una estructura entre **pila**, **cola**, **lista simple**, **lista circular** y **lista doble**. Explica qué operación o propiedad de la estructura hace que sea adecuada.

1. Un editor de texto guarda las acciones recientes para permitir deshacer primero la última acción realizada.
   R// **Pila (Lifo)** ya que guarda la acción realizada recientemente paraa que pueda ser la primera que se pueda revertir.
2. Las personas esperan su turno en una fila de atención y se atiende primero a quien llegó primero.

R// **Cola (Fifo)** ya que el primero en llegar es el que lleva mas tiempo y por ende es el primero a él que atienden, primero en llegar es el primero en salir

3. Una aplicación recorre las canciones de una lista y, al terminar la última, vuelve automáticamente a la primera.
   R// **Lista circular** ya que no tiene un fin definitivo sino que al llegar al supuesto final esta se vuelve a reproducir ya que si final está conectado con el principio lo que sería el null y el head.
4. Un sistema necesita recorrer una colección de registros en un único sentido y agregar elementos al comienzo con frecuencia.
   R// **Lista simple** ya que solo tiene una direccion y solo requiere actualizar unicamente la cabeza.
5. Un reproductor permite pasar a la canción siguiente y a la anterior desde cualquier canción.
   R// **Lista doblemente enlazada** ya que cada nodo debe tener dos referencias (siguiente y anterior) para que permita hacer la acción que se necesita y permita ir en ambas direcciones.

Parte 2 — Pilas (LIFO)

Una pila sigue regla **último en entrar, primero en salir**. Considera que `push(x)` agrega `x` al tope, `pop()` retira y devuelve el elemento del tope, y `peek()` consulta el tope sin retirarlo.

2.1 Sigue las operaciones

La pila inicia vacía. Ejecuta las operaciones en orden y dibuja el contenido después de cada una. Indica el resultado de cada `pop()` y de cada `peek()`.

push(4): **[4 (tope)]**

push(9): **[9 (tope), 4]**

push(2): **[2 (tope), 9, 4]**

pop(): **Retira 2. Queda: [9 (tope), 4]**

push(7): **[7 (tope), 9, 4]**

peek(): **Consulta el tope sin retirarlo. Resultado: 7**

pop(): Retira 7. **Queda: [9 (tope), 4]**

pop(): Retira 9. **Queda: [4 (tope)]**

Al terminar, indica qué elemento queda en la pila y cuál es el tope. Usa un dibujo que deje claro cuál extremo corresponde al tope.

RESULTADO: el elemento es 4 el cual también es el tope.



2.2 Casos límite

1. ¿Qué debería ocurrir si se ejecuta `pop()` sobre una pila vacía? Describe una forma de manejarlo.
   R// Aparece algo llamado desbordamiento por subflujo, una manera de controlarlo es poner que nos indique que hay un error, por ejemplo "Error: Pila vacía".
2. ¿Qué debería devolver `peek()` cuando la pila está vacía? Describe una forma de manejarlo.
   R// No hay ningún tope que consultar, y se puede manejar lanzando un mensaje de advertencia que nos informe la ausencia de datos.
3. Escribe pseudocódigo para una operación `deshacer(pila)` que retire y devuelva la última acción, controlando el caso de pila vacía.
   R//

```
Función deshacer (pilaAcciones)
  SI pilaAcciones.estaVacia( ) ENTONCES
     RETORNAR "No hay acciones para deshacer"
  FIN SI
  RETORNAR pilaAcciones.pop ( )
FIN FUNCIÓN
```

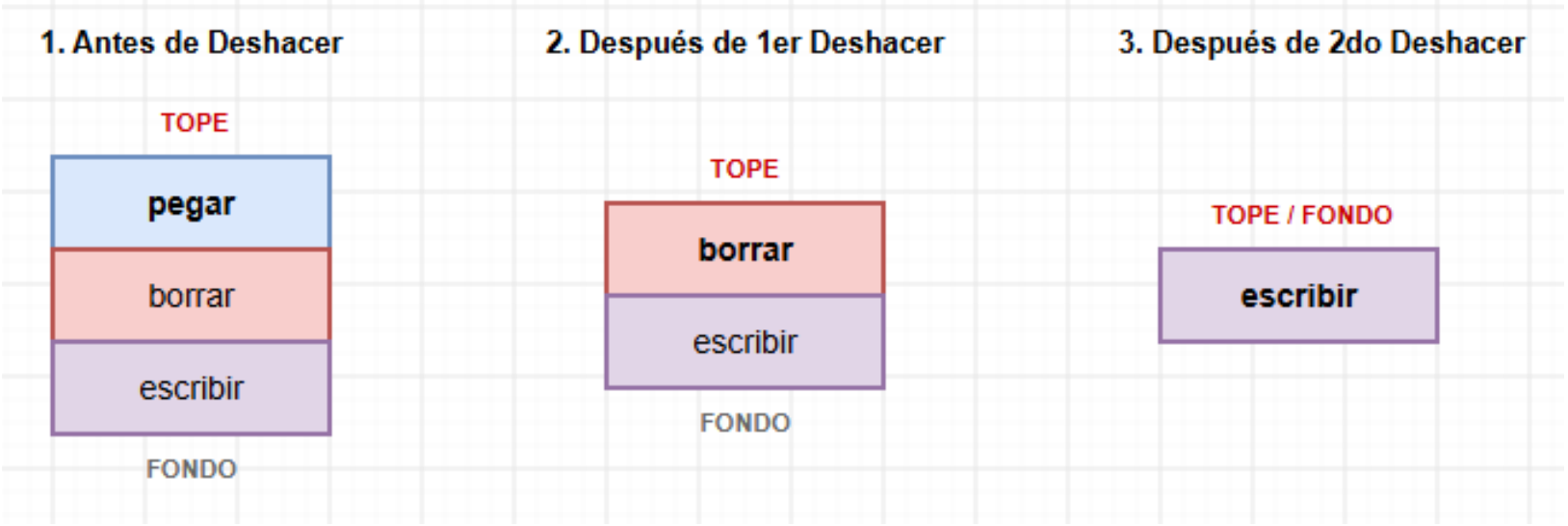
2.3 Aplicación: deshacer acciones

Un usuario realiza, en este orden, las acciones `escribir`, `borrar` y `pegar`. Luego presione "deshacer" dos veces.

- Dibuja la pila de acciones antes y después de cada deshacer.

- ¿Qué acción se deshace primero? Explica por qué una pila representa este comportamiento.

R//



Parte 3 — Colas (FIFO)

Una cola sigue la regla **primero en entrar, primero en salir**. En esta actividad, `encolar(x)` agrega al final y `desencolar()` retira desde el frente.

3.1 Simula la fila de atención

La cola empieza vacía. Ejecuta las operaciones en orden y dibuja la cola después de cada operación, marcando **frente** y **final**. Anota qué persona devuelve cada `desencolar()`.

encolar("Ana"): **Frente: [Ana], Final: [Ana]**

encolar("Luis"): **Frente: [Ana], Final: [Luis]**

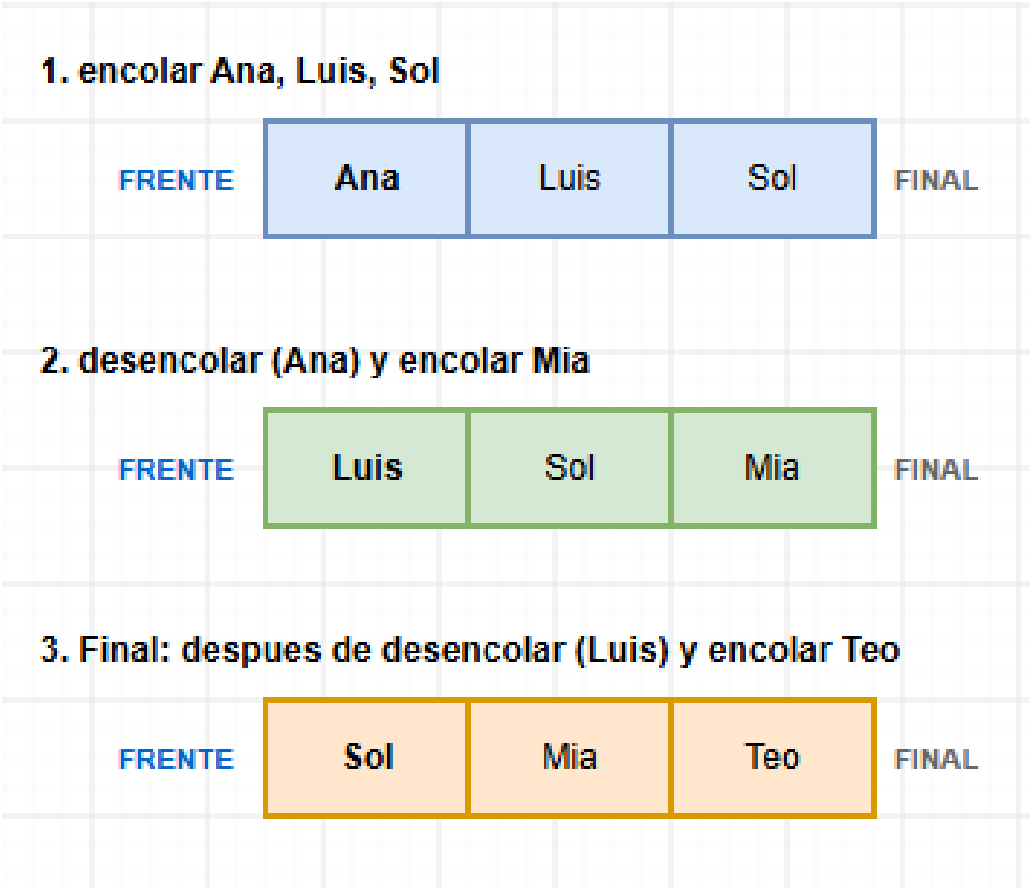
encolar("Sol"): **Frente: [Ana, Luis], Final: [Sol]**

desencolar(): **Sale: "Ana". Quedan: Frente [Luis], Final [Sol]**

encolar("Mia"): **Frente [Luis, Sol], Final [Mia]**

desencolar(): **Sale: "Luis". Quedan: Frente [Sol], Final [Mia]**

encolar("Teo"): **Frente [Sol, Mia], Final [Teo]**



¿Qué personas quedan esperando y en qué orden?
R// del frente al final (SOL, MIA, TEO)

3.2 Operaciones y casos límite

1. Escribe pseudocódigo para `atender(cola)`, que retire y devuelva a la persona del frente.
   R//
   Función atender(cola)
   SI cola.estaVacia ( ) ENTONCES
   RETORNAR "no hay personas en espera"
   FIN SI
   RETORNAR cola.desencolar ( )
   FIN FUNCIÓN
2. ¿Qué debería ocurrir si se intenta desencolar una cola vacía? Describe cómo lo manejarías.

R// Ocurre un subflujo de cola que se maneja validando si el frente es nulo o si el tamaño es 0, y se retorna un mensaje de error controlado para que el sistema no falle.

3. Explica por qué agregar al frente y retirar desde el final no representa correctamente una fila FIFO si se mantienen esas mismas operaciones.
   R// porque si el que llega de ultimo saldria primero, esto convertiría nuestra estructura de cola en una pila, ya que esto sería LIFO

Parte 4 — Listas enlazadas simples

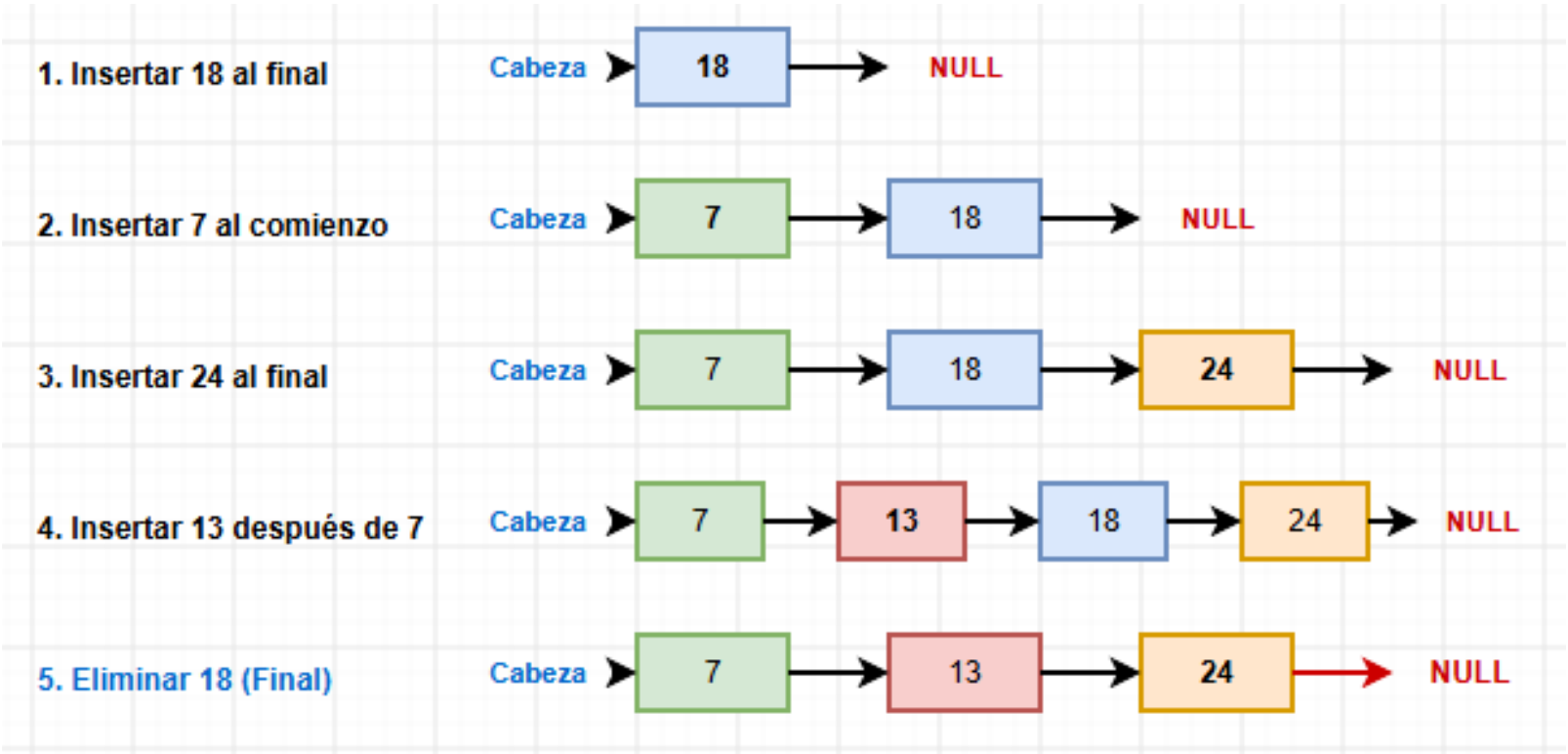
En una lista simple, cada nodo guarda un dato y una referencia `siguiente`. El último nodo apunta a `nulo`.

4.1 Construye y modifica una lista

Parte de la lista vacía y aplica en orden estas operaciones:

1. Insertar `18` al final: **[18] → NULL**
2. Insertar `7` al comienzo: **[7] → [18] → NULL**
3. Insertar `24` al final: **[7] → [18] → [24] → NULL**
4. Insertar `13` después del nodo que contiene `7`: **[7] → [13] → [18] → [24] → NULL**
5. Eliminar el nodo que contiene `18`: **[7] → [13] → [24] → NULL (El nodo final apunta a NULL).**

Dibuja la lista después de cada operación. En el último dibujo, muestra explícitamente que el nodo final apunta a `nulo`.



4.2 Diseña operaciones

Escribe pseudocódigo para:

1. Recorrer la lista e imprimir todos sus datos.
   R//
   Función Recorrer (actual, cabeza)
   actual = cabeza
   MIENTRAS actual != NULL hacer
   IMPRIMIR (actual.dato)
   actual = actual.siguiente
   FIN MIENTRAS
   FIN Función
2. Buscar un valor y devolver si está presente.
   R//
   FUNCION Buscar (cabeza, valorBuscado) bool
   actual = cabeza

   MIENTRAS actual != Nulo
   SI actual == valorBuscado
   ENTONCES True
   FIN SI

   SINO actual = actual.siguiente
   FIN MIENTRAS

   RETORNAR false
   FIN FUNCION
3. Insertar un nodo al comienzo, actualizando correctamente la referencia `cabeza`.
   R//
   FUNCION InsertarInicio (cabeza,valor) -> NODO
   NuevoNodo <- CREAR Nodo
   NuevoNodo.dato <- valor
   nuevoNodo = cabeza
   cabeza = NuevoNodo
   RETORNAR cabeza
   FIN FUNCION

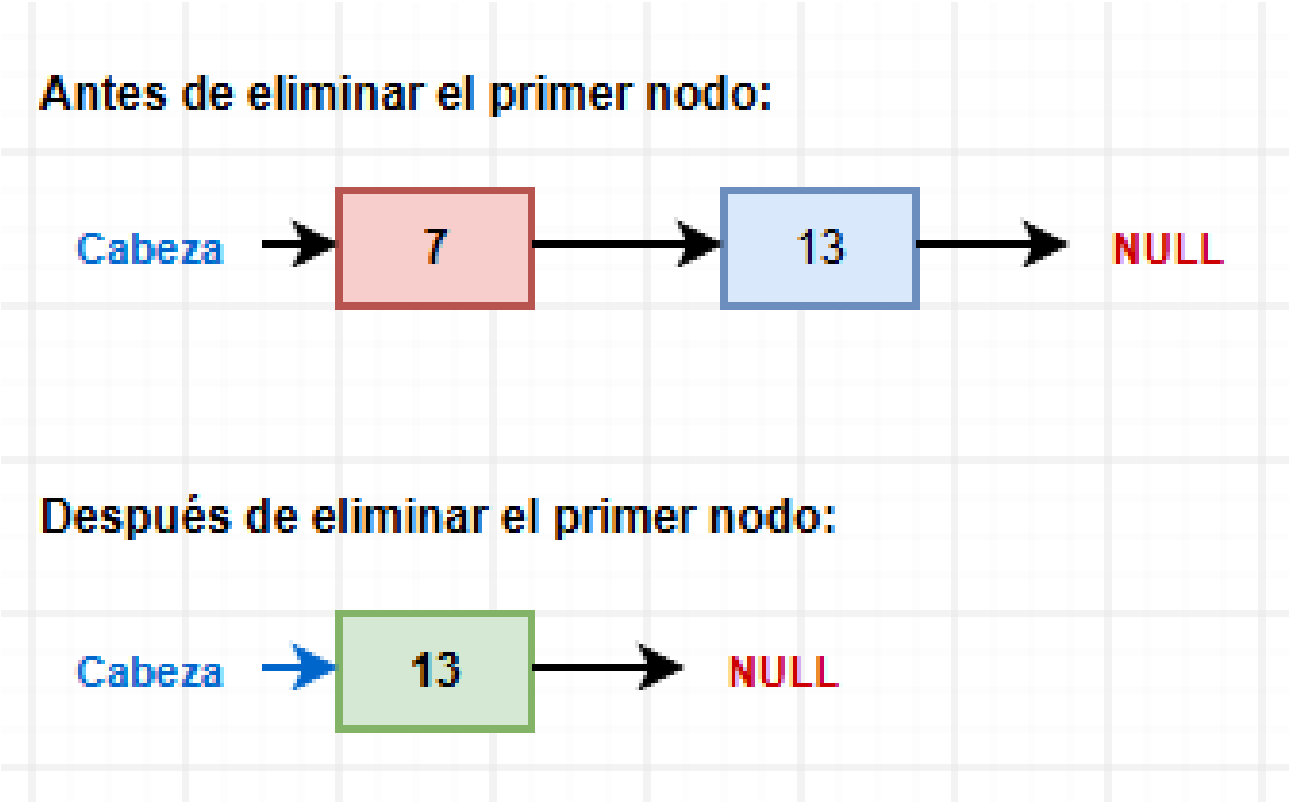
4.3 Comprueba los bordes

Explica cómo se comporta tu operación de insertar al comienzo cuando:

- La lista está vacía.
  R// Se crea un nuevo nodo con el valor indicado. La referencia cabeza (que inicialmente apuntaba a null) pasa a apuntar directamente a este nuevo nodo. El enlace siguiente de este nodo apunta a null, ya que es el único elemento.

- La lista ya tiene uno o más nodos.
  R// Se crea un nuevo nodo. Su enlace siguiente se configura para que apunte al nodo que actualmente ocupa la primera posición (al que apuntaba cabeza). Finalmente, se actualiza la referencia cabeza para que apunte al nuevo nodo creado.

¿Qué referencias deben actualizarse al eliminar el primer nodo? Apóyate con un diagrama antes y después.
R// Al eliminar el primer nodo, la referencia cabeza debe actualizarse para que apunte al segundo nodo de la lista (es decir, cabeza.siguiente). Despues, se libera el espacio en memoria del nodo antiguo.



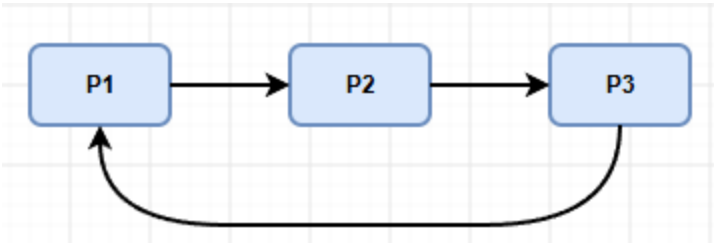
Parte 5 — Listas circulares

En una lista circular simple, cada nodo tiene una referencia `siguiente`, pero el último nodo apunta de nuevo al primero. No hay un enlace final a `nulo`.

5.1 Reproduce un recorrido circular

Tres procesos, P1, P2 y P3, esperan recibir turnos en una lista circular. Cada turno dura una unidad de tiempo. Se atiende un proceso por turno; al terminar su turno, si sigue activo, se pasa al siguiente. P2 termina en el segundo turno que recibe.

1. Dibuja la lista circular inicial, incluyendo el enlace del último nodo al primero.
   R//



2. Registra los primeros siete turnos: indica qué proceso recibe cada uno y cuál será el siguiente.
   R//

Turno 1: Se atiende a **P1**. Siguiente proceso en recibir turno: P2.
Turno 2: Se atiende a **P2** (recibe su segundo turno y termina). Siguiente proceso: P3.
Turno 3: Se atiende a **P3**. Siguiente proceso: P1.
Turno 4: Se atiende a **P1**. Siguiente proceso: P3 (ya que P2 fue retirado).
Turno 5: Se atiende a **P3**. Siguiente proceso: P1.
Turno 6: Se atiende a **P1**. Siguiente proceso: P3.
Turno 7: Se atiende a **P3**. Siguiente proceso: P1.

Al eliminar P2, el nodo de P1 ahora apunta directamente a P3, y el nodo de P3 mantiene su enlace apuntando de vuelta a P1, cerrando el ciclo perfectamente.

3. Dibuja la lista después de retirar P2. Verifica que el último nodo siga apuntando al primero.
   R//



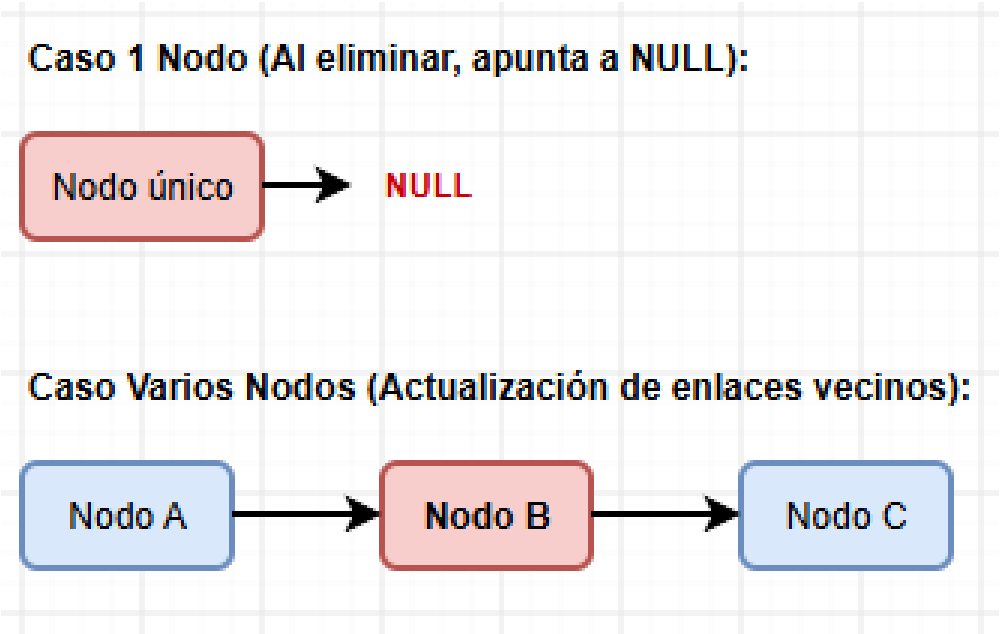
4. ¿Qué condición usarías para detener un recorrido de todos los nodos, dado que no se llega a `nulo`?
   R// Ya que no existe un puntero null, la condición adecuada consiste en almacenar en una variable el nodo inicial donde comenzó la búsqueda (nodoInicio) y detener el ciclo (usando un do-while) cuando el nodo actual vuelva a ser igual a nodoInicio después de haber avanzado al menos una vez.

5.2 Casos especiales

Explica qué enlaces se deben actualizar al eliminar un nodo en cada caso:

1. La lista está vacía.
   R// No hay nodos; se retorna un aviso de error o lista vacía sin alterar referencias.
2. La lista contiene un solo nodo.
   R// Se elimina el nodo y la referencia principal de inicio (cabeza o inicio) pasa a apuntar a null.
3. Se elimina el nodo que actualmente se considera primero.
   R// Se actualiza la referencia de inicio (cabeza) para que apunte al segundo nodo, y se recorre hasta el último nodo para actualizar su enlace siguiente de modo que apunte al nuevo primer nodo.
4. Se elimina el último nodo.
   R// Se recorre la lista hasta el penúltimo nodo y se actualiza su enlace siguiente para que apunte al primer nodo de la lista.

Incluye un pequeño diagrama para los casos de un nodo y de dos o más nodos.



Parte 6 — Listas doblemente enlazadas

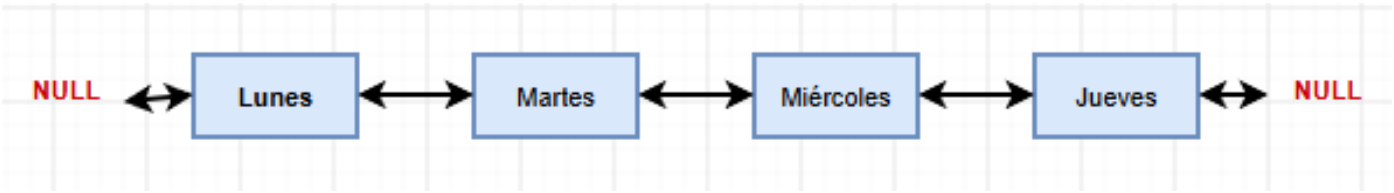
En una lista doble, cada nodo tiene referencias `anterior` y `siguiente`. El primer nodo no tiene anterior (`nulo`) y el último no tiene siguiente (`nulo`).

6.1 Recorrido en ambos sentidos

Construye esta lista:

Lunes ⇄ Martes ⇄ Miércoles ⇄ Jueves

1. Dibuja cada nodo con su dato, su enlace `anterior` y su enlace `siguiente`.
   R//



2. Recorre la lista desde el comienzo hasta el final y escribe el orden de los datos.
   R// Lunes → Martes → Miércoles → Jueves.
3. Recorre desde el final hasta el comienzo y escribe el orden inverso.
   R// Jueves → Miércoles → Martes → Lunes.

6.2 Inserta y elimina

Parte de la lista anterior.

1. Inserta `Domingo` antes de `Lunes`.
   R// El nodo **Domingo** se coloca al inicio; su puntero **anterior** apunta a **NULL** y su **siguiente** a **Lunes**. El enlace **anterior** de **Lunes** se actualiza a **Domingo**.
2. Inserta `Viernes` después de `Jueves`.
   R// El nodo **Viernes** se añade al final; su puntero **anterior** apunta a **Jueves** y su **siguiente** a **NULL**. El enlace **siguiente** de **Jueves** se actualiza a **Viernes**.
3. Elimina `Miércoles`.
   R// El nodo anterior (**Martes**) actualiza su enlace **siguiente** para que apunte directamente a **Jueves**. El nodo posterior (**Jueves**) actualiza su enlace **anterior** para que apunte a **Martes**.

Dibuja el resultado después de cada operación. Para la eliminación, señala qué referencias `anterior` y `siguiente` cambian en los nodos vecinos.



6.3 Compara los enlaces

En 3–5 líneas, explica una ventaja y un costo de usar una lista doble en lugar de una lista simple. Usa el recorrido hacia atrás del ejercicio anterior en tu explicación.

R// Una gran **ventaja** de la lista doblemente enlazada es que nos permite hacer recorridos y búsquedas en las dos direcciones (hacia adelante y hacia atrás) de manera eficiente desde cualquier nodo, haciendo mucho más fácil operaciones inversas (como recorrer del fin al inicio) sin requerir punteros auxiliares externos. El principal **costo** es el consumo adicional de memoria por cada nodo ya que estos almacenan un puntero extra (anterior).

Parte 7 — Análisis y elección

Completa la tabla. Indica **O(1)** u **O(n)** bajo los supuestos de la columna “Representación”. Si necesitas hacer un supuesto adicional, escríbelo junto a la respuesta.

Estructura y representación	**Operación**	**Complejidad**	**Justificación breve**
Pila enlazada con referencia al tope	Insertar en el tope	$O(1)$	Solo se crea un nodo y se ajusta el puntero de arriba directamente.
Pila enlazada con referencia al tope	Retirar del tope	$O(1)$	Se actualiza la referencia del tope al nodo siguiente en tiempo constante.
Cola enlazada con referencias al frente y al final	Encolar al final	$O(1)$	Gracias al puntero directo al final, el nuevo elemento se inserta de inmediato.

Estructura y representación	**Operación**	**Complejidad**	**Justificación breve**
Cola enlazada con referencias al frente y al final	Desencolar del frente	$O(1)$	Se quita el nodo del frente y se pasa a la referencia del frente de forma directa.
Lista simple sin referencia al último nodo	Insertar al comienzo	$O(1)$	Se enlaza el nuevo nodo directamente con la cabeza actual.
Lista simple sin referencia al último nodo	Insertar al final	$O(n)$	Se debe recorrer secuencialmente toda la lista desde la cabeza hasta el final.
Lista doble con referencia a un nodo conocido	Desconectar ese nodo	$O(1)$	tiene referencias directas a sus vecinos (anterior y siguiente), permitiendo reasignarlos de inmediato.

Responde además:

1. ¿Por qué una cola enlazada que solo guarda la referencia al frente podría tardar más en encolar al final que una que también guarda la referencia al último nodo?
   R// Porque al no tener un puntero directo al final, es obligatorio recorrer toda la lista nodo por nodo desde el frente, lo que toma un tiempo proporcional al tamaño ($O(n)$) en lugar de hacerse de manera directa ($O(1)$).

2. Para una aplicación que necesita volver repetidamente al primer elemento al llegar al final, ¿qué ventaja ofrece la lista circular?
   R// Da la ventaja de que el último nodo se conecta con el primero mediante su enlace siguiente, permitiendo transiciones continuas sin pedir verificaciones de reinicio manual.

3. ¿Qué información adicional almacena una lista doble frente a una simple, y qué permite hacer esa información?
   R// guarda un puntero o referencia anterior en cada nodo (además del dato y el puntero siguiente), lo que permite navegar y recorrer la estructura de forma bidireccional (hacia adelante y hacia atrás).

Parte 8 — Reflexión

En un párrafo de 5 a 8 líneas, explica cómo se relacionan la forma de conectar los nodos y el orden en que se insertan o retiran datos. Menciona al menos tres de estas estructuras: pila, cola, lista simple, lista circular y lista doble.

R// Básicamente, la forma en que conectamos los nodos define cómo se mueven y se guardan los datos. Por ejemplo, en una pila los enlaces solo dejan operar en el tope, cumpliendo con el LIFO, mientras que las colas acomodan los elementos en fila para atender siempre al primero que llega (FIFO). Por otro lado, las listas muestran lo facil y que los punteros pueden hacer de todo. las circulares pegan el último nodo con el primero para armar un ciclo continuo, y las dobles conectan los dos lados para ir hacia adelante y hacia atrás sin problema. Al final, la arquitectura de los enlaces es la que manda sobre cómo responde cada estructura.

Criterios de evaluación

Criterio	**Peso**
Comprensión de las reglas de cada estructura	20%
Trazas y diagramas correctos, con referencias identificadas	25%
Pseudocódigo y actualización de enlaces	25%
Manejo de casos límite y análisis de complejidad	20%
Claridad, orden y explicación propia	10%

Se evalúa el razonamiento y la justificación, además de los resultados.